Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 6:
Client UDP Multicast

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Istruzioni di Esecuzione 4

4 Risultati **4**
4.1 Istanziare molteplici Client 4
4.2 Analisi Wireshark 5

5 Conclusioni e Sviluppi Futuri **6**

1 Problem Statement

L'obiettivo della sesta esercitazione (Hands-On 6) è lo sviluppo di un client UDP multicast in linguaggio C. Il client deve potersi unire a un gruppo multicast al fine di ricevere i messaggi del server fornito dal Docente (`multicast_server.c`). Viene richiesto inoltre che sia possibile lanciare molteplici istanze del client in parallelo per ricevere i messaggi simultaneamente.

2 Stato dell'Arte

La comunicazione multicast è di tipo uno-a-molti: un host invia un messaggio una singola volta e la rete, utilizzando protocolli come all'IGMP (Internet Group Management Protocol), si assicura di replicarlo e recapitarlo a tutti gli host iscritti.

A livello di sistema (socket API), la gestione del multicast avviene tramite una serie di direttive con cui il processo informa il kernel della propria volontà di inoltrare la sottoscrizione al gruppo. Le procedure implementate sono le seguenti:

- **SO_REUSEADDR**: Modificare le opzioni originarie del socket affinchè il sistema permetta il collegamento di *bound* sulla medesima porta e lo stesso IP a più processi.
- **Binding su INADDR_ANY**: Necessario per catturare i pacchetti dalla scheda di rete che transitano sul gruppo al quale ci sottoscriviamo.
- **IP_ADD_MEMBERSHIP**: Direttiva tramite cui il socket comunica l'intenzione di aggregarsi ad uno specifico indirizzo IP multicast.

3 Metodologia ed Implementazione

Il codice sorgente per il multicast si basa su due strutture principali e sull'uso mirato di `setsockopt()`.

1 e 2: Creazione Socket e SO_REUSEADDR

```c
int sockfd;
if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}

int reuse = 1;
if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &reuse,
    sizeof(reuse)) < 0) {
    perror("SO_REUSEADDR failed");
    exit(EXIT_FAILURE);
}
```

Si effettua quindi il bind e si provvede all'iscrizione al gruppo Multicast (nel nostro caso `239.0.0.1`).

3 e 4: Bind e Add Membership

```
// Bind
local_addr.sin_family = AF_INET;
local_addr.sin_port = htons(PORT);
local_addr.sin_addr.s_addr = INADDR_ANY;
bind(sockfd, (struct sockaddr *)&local_addr,
    sizeof(local_addr));

// Iscrizione Multicast
struct ip_mreq multicast_req;
multicast_req.imr_multiaddr.s_addr = inet_addr("239.0.0.1");
multicast_req.imr_interface.s_addr = INADDR_ANY;

setsockopt(sockfd, IPPROTO_IP, IP_ADD_MEMBERSHIP,
    &multicast_req, sizeof(multicast_req));
```

Nel loop principale si esegue in ultimo la normale `recvfrom()` bloccante per ricevere il payload su pacchetti inviati in UDP.

5: Ricezione dei messaggi UDP

```
while (1) {
    int n = recvfrom(sockfd, buffer, MAX_BUFFER - 1, 0,
                     (struct sockaddr *)&sender_addr,
                        &sender_len);
    // ... logic per stampa e gestione terminazione ...
}

// 6: Dropout in dismissione
setsockopt(sockfd, IPPROTO_IP, IP_DROP_MEMBERSHIP,
    &multicast_req, sizeof(multicast_req));
```

3.1 Istruzioni di Esecuzione

Compilazione della suite multicast:

Multicast Build

```
$ gcc multicast_server.c -o server
$ gcc multicast_client.c -o client
```

4 Risultati

Per validare il comportamento del software, sono stati testati esecutivamente due processi client separati ed un unico mittente (il server multicast) sulla medesima subnet.

4.1 Istanziare molteplici Client

I log di terminale sottostanti dimostrano come il kernel riceva il traffico multicast unificato, recapitando lo stream del buffer "Hello, Multicast!" ad ogni terminale.

Terminale 1 - Server

```
$ ./server
Sent: Hello, Multicast!
Sent: Hello, Multicast!
Sent: Hello, Multicast!
```

Terminale 2 - Client 1

```
$ ./client
Client iscritto al gruppo multicast 239.0.0.1 sulla porta
   12345
In attesa di messaggi...

Ricevuto: Hello, Multicast! | (Da: 127.0.0.1:46155)
Ricevuto: Hello, Multicast! | (Da: 127.0.0.1:46155)
```

Terminale 3 - Client 2

```
$ ./client
Client iscritto al gruppo multicast 239.0.0.1 sulla porta
   12345
In attesa di messaggi...

Ricevuto: Hello, Multicast! | (Da: 127.0.0.1:46155)
Ricevuto: Hello, Multicast! | (Da: 127.0.0.1:46155)
```

Entrambi i socket UDP sono stati abilitati alla convivenza sullo stesso port listening e intercettano con precisione i messaggi mandati in overlay sull'IP appartenente al gruppo di classe D.

4.2 Analisi Wireshark

L'analisi del traffico tramite Wireshark (Figura 1) conferma il corretto inoltro dei pacchetti UDP verso l'indirizzo di classe D `239.0.0.1`. Si osserva come un singolo pacchetto inviato dal server venga ricevuto da tutti i client iscritti al gruppo, minimizzando il carico di rete rispetto ad una comunicazione unicast multipla.

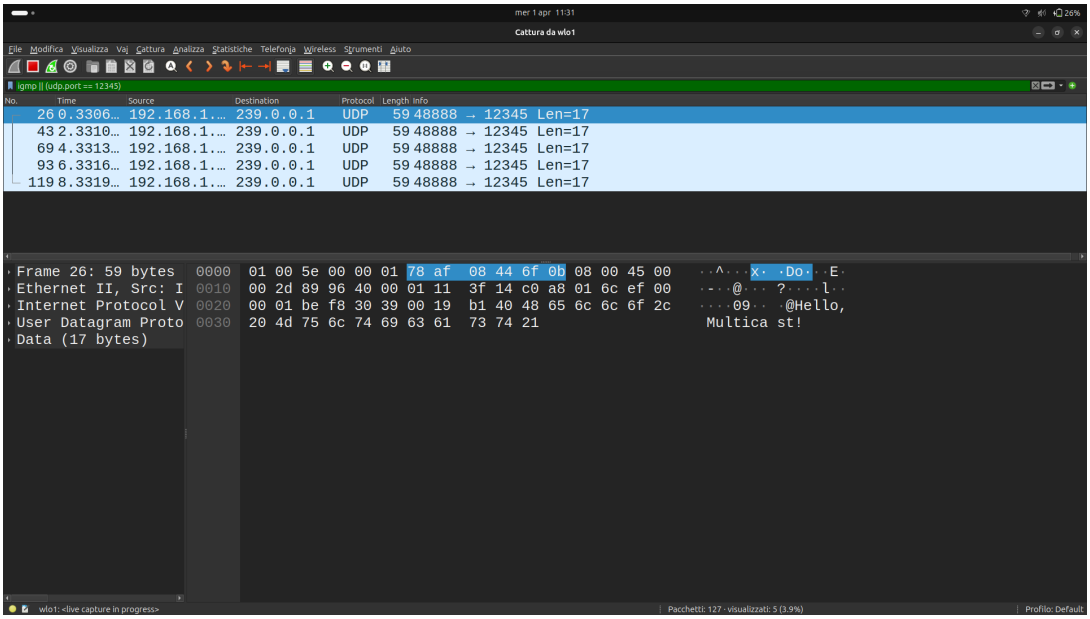

Figure 1: Cattura Wireshark: Flusso Multicast UDP verso 239.0.0.1

5 Conclusioni e Sviluppi Futuri

Durante l'Hands-On 6 è stato analizzato l'assetto implementativo richiesto per abilitare flussi uno-a-molti nelle architetture basate su datagrammi in linguaggio C, superando in modo efficace i vincoli stringenti dettati dall'API standard ed approfondendo i benefici dell'opzione estesa SO_REUSEADDR e l'invio degli intent IGMP via syscall.